# PYTHON NOTES

**Akhil Tangudu**




## INTRODUCTION

Python is high level, interpreted, interactive and object oriented programming language

# CONTENT

**Unit-1:**

Introduction to Python

**Unit-2:**

Data Structures and Sequences

**Unit-3:**

File Handling in Python

**Unit-4:**

Topic-1 Numpy

Topic-2 Pandas

**Unit-5:**

Matplotlib

# UNIT-01 INTRODUCTION TO PYTHON

**Q1) What is Python?**

**Ans)** Definition: Python is a high-level, interpreted, interactive and object-oriented scripting language.

**Python is a high-level programming language.**

This means it is easy for humans to read and write compared to many other programming languages.

**Python is an interpreted language.**

The Python interpreter runs the program **line by line at runtime**, so you do not need to compile the program before running it.

**Python is an interactive language.**

You can type Python commands directly in the Python prompt and see the result immediately.

**Python supports object-oriented programming.**

This means programs can be organized using **objects**, which combine data and functions together.

**Python is designed to be very readable.**

It uses **English words (keywords)** like if, else, and while, instead of many special symbols.

**Python has simple syntax.**

It uses fewer complicated rules compared to many other programming languages.

**Python is beginner-friendly.**
It is a very good language for people who are learning programming for the first time.

---

## Q2) What are the features of python?

**(or)**

## What is the need for Python Programming?

**Ans)**Python has many features that make it a popular programming language.

1. Free and Open Source

Python is free to use. Anyone can download it, use it, modify it, and share it.

2. Simple

Python is a simple language. Its syntax is easy and similar to English, so programs are easy to read and write.

3. Easy to Learn

Python has fewer keywords and simple rules. Beginners can learn it quickly.

4. Easy to Read

Python code is clear and well structured, so it is easy for programmers to understand.

5. Easy to Maintain

Python programs can be easily modified and maintained when needed.

6. Versatile

Python can be used to develop many types of applications such as text processing, web applications, games, and scientific software.

## 7. High-Level Language

Python is a high-level language, which means programmers do not need to manage memory or hardware details.

## 8. Object-Oriented

Python supports object-oriented programming concepts such as classes and objects.

## 9. Interpreted Language

Python programs are executed line by line by the interpreter. No compilation is required before running the program.

## 10. Large Standard Library

Python provides many built-in modules and libraries that help programmers perform tasks easily.

## 11. Interactive Mode

Python supports an interactive mode where programmers can test code directly in the interpreter.

## 12. Portable

Python programs can run on different platforms such as Windows, Linux, and Mac without major changes.

## 13. Extendable

Python allows programmers to add low-level modules written in other languages like C or C++.

### 14. Database Support

Python provides interfaces to connect with many databases.

### 15. GUI Programming

Python supports graphical user interface (GUI) development.

### 16. Multi-threading

Python can execute multiple tasks at the same time.

### 17. Scalable

Python supports development of both small and large programs.

### 18. Robust

Python automatically handles memory and provides clear error messages.

---

## Q3) What are Applications of Python?

**Ans)** Python has many applications

1. Web Development
2. Data Science
3. Machine Learning
4. Artificial Intelligence
5. Automation
6. Scripting
7. Game Development

8. Cyber Security
9. IoT

---

## Q4) What are the Limitations of Python?

# Ans)Limitations of Python

**1. Speed**

Python programs run **slower than languages like C and C++.**
This is because Python executes code **line by line using an interpreter.**

**2. Mobile Development**

Python is **not commonly used for developing mobile applications.**
Because of this, only **a few mobile apps are built using Python.**

**3. Memory Consumption**

Python uses **more memory compared to some other programming languages.**
This happens because Python data types are very flexible.

**4. Database Access**

Python can connect to databases, but its **database tools are not as advanced** as technologies like **JDBC and ODBC.**
So it may **not be the best choice for very large enterprise systems that use complex databases.**

**5. Runtime Errors**

Python is a **dynamically typed language**, which means the data type is checked while the program is running.

Because of this, **some errors appear only when the program is executed**, so programmers need to test the program carefully.

---

## Q5) Explain about Comment lines in Python?

### Ans) Python Comments

- **Comments** are lines written inside a program to explain the code.
- The **Python interpreter ignores comments**, so they are not executed.
- Comments help **make the code easier to read and understand.**
- They help programmers explain **what the program is doing.**

# Types of Comments in Python

There are **three types of comments** in Python:

- Single-line comments
- Multi-line comments
- Docstring comments

### 1.) Single-Line Comments

- **A single-line comment starts with the symbol #.**
- Everything written **after # on that line is treated as a comment.**

- It continues **until the end of the line.**
- These comments are usually used to **give short explanations.**

**Example**

```
# Hello World Demo

print("Hello World")
```

## 2. )Multi-Line Comments

- Python **does not have a special symbol for multi-line comments.**
- But we can write multi-line comments in two ways.

**Method 1: Using Multiple #**

- We can write **# at the beginning of each line.**
- Each line will be treated as a separate single-line comment.

Example:

```
# Python program to demonstrate

# multiline comments

print("Multiline comments")
```

Output:

Multiline comments

**Method 2: Using String Literals**

- Python ignores **strings that are not assigned to a variable.**

- Because of this, we can use **triple quotes (""" """)** to write multi-line comments.

Example:

""" Python program to demonstrate

multiline comments """

print("Multiline comments")

Output:

Multiline comments

### 3.) Python Docstring

- A **docstring** is a special type of comment written using **triple quotes (""" """).**
- It is written **just below a function, class, or module.**
- It is used to **explain what the function or program does.**
- Docstrings help programmers **understand the purpose of the code.**
- The docstring can be accessed using the **__doc__ attribute** in Python.

## Example

```
def multiply(a, b):
    """Multiplies the value of a and b"""
    return a * b

# Print the docstring of the function
print(multiply.__doc__)
```

**Output**

Multiplies the value of a and b

---

## Q6) Explain about Variables in Python.

## **Ans)** Variables in Python

- Variables are used to store data in a program.
- When you create a variable, the computer reserves some memory space to store the value.
- A variable is simply a name that refers to a value.

Example:

a = 10

name = "Akhil"

Here:

- a stores the value 10
- name stores the value "Akhil"

## Objects in Python

In Python, variables refer to objects.
Every object has three things:

1. Identity

- Identity means the memory address of the object.
- It does not change once the object is created.
- We can find it using the id() function.

Example:

x = 10

print(id(x))

2. Type

- Type means the data type of the object.
- It tells what kind of value is stored.

Examples of types:

- int → integer numbers
- float → decimal numbers
- str → text

Example:

x = 10

print(type(x))

## 3. Value

- The value is the actual data stored in the variable.
- It can be a number, text, or other data.

Example:

x = 5

Here:

- Variable → x
- Value → 5

## Rules for Creating Variables in Python

When naming variables, follow these rules:

- A variable name must start with a letter or underscore _.
- A variable name cannot start with a number.
- A variable name can contain letters, numbers, and underscores.
- Variable names are case-sensitive.

Example:

name, Name, NAME

These are three different variables.

- Keywords cannot be used as variable names.

Example (invalid):

class = 10

## Assigning Values to Variables

- Python does not require declaring variables before using them.
- A variable is created automatically when a value is assigned.
- The assignment operator = is used to store values in variables.

Example:

a = 100

avg = 75.3

name = "John"

print(a)

print(avg)

print(name)

## Multiple Assignment to Variables

Python allows assigning one value to many variables at the same time.

Example:

a = b = c = 1

Here:

- a, b, and c all store the value 1.

We can also assign different values to different variables in one line.

Example:

a, b, c = 1, 2.5, "KMIT"

Here:

- a → 1
- b → 2.5
- c → "KMIT"

Example program:

x = y = z = 0

print(x)

print(y)

print(z)

Output:

0

0

0

## Deleting Variables in Python

Sometimes we may want to remove a variable from memory after using it.

There are two ways to do this.

### 1. Assign None to the Variable

- None means the variable has no value.
- The variable name still exists, but it contains no data.

Example:

queue = None

### 2. Delete the Variable Using del

- The del keyword removes the variable completely.
- After deleting, the variable cannot be used again.

Example:

del queue

Example:

a = 1

```
print(a)


a = None
print(a)


del a
print(a)
```

Output:

```
1
None
NameError: name 'a' is not defined
```

---

## Q7) Explain about Keywords in Python.

**Ans)** Keywords in Python

Keywords are reserved words in Python that have a special meaning. They are used to perform specific tasks in a program.

- Keywords are predefined by Python.
- They cannot be used as variable names, function names, or identifiers.
- They help define the structure and logic of a Python program.
- All keywords are case-sensitive.

Example: if is a keyword used for decision making.

```
if x > 10:
    print("Greater than 10")
```

Here, if is a keyword that checks a condition.

### Examples of Python Keywords

Some commonly used Python keywords are:

```
False   await   else    import   pass
None    break   except  in       raise
True    class   finally is       return
and     continue for    lambda   try
as      def     from    nonlocal while
assert  del     global  not      with
async   elif    if      or       yield
```

### Examples of Keywords in Programs

#### Example 1 – if keyword

```
x = 5
if x > 3:
    print("x is greater than 3")
```

Explanation:
if checks whether the condition is true.

Example 2 – for keyword

```
for i in range(3):
    print(i)
```

Explanation:
for is used to repeat a block of code.

Example 3 – def keyword

```
def greet():
    print("Hello")
```

Explanation:
def is used to define a function.

Important Points

- Keywords have fixed meanings in Python.
- They cannot be used as identifiers.
- They help control program flow, loops, conditions, and functions.

Example (Invalid):

```
class = 10
```

This will cause an error because class is a keyword.

✅ Short Exam Definition

Keywords are reserved words in Python that have predefined meanings and cannot be used as variable names or identifiers.

---

## Q8) Explain about Data types in Python.

## Ans) 🐍 Data Types in Python

A data type defines the type of value a variable can store.

In Python, data types are mainly divided into:

1. Numeric Types
2. Sequence Types
3. Set Types
4. Mapping Type
5. Boolean Type
6. None Type

1️⃣ Numeric Data Types

Used to store numbers.

- 1. int (Integer)

Stores whole numbers (positive or negative).

x = 10

y = -5

print(type(x))  # <class 'int'>

- 2. float

Stores decimal numbers.

a = 3.14

b = -2.5

print(type(a))  # <class 'float'>

🔹 3. complex

Stores complex numbers (real + imaginary).

z = 2 + 3j

print(type(z))  # <class 'complex'>

2️⃣ Sequence Data Types

Used to store multiple items in an ordered way.

🔹 1. str (String)

Stores text (inside quotes).

name = "Akhil"

print(type(name))  # <class 'str'>

✔ Strings are immutable (cannot be changed).

🔹 2. list

Stores multiple items in square brackets [ ].

numbers = [1, 2, 3, 4]

print(type(numbers))  # <class 'list'>

✔ Lists are ordered and mutable (can be changed).

🔹 3. tuple

Stores multiple items in round brackets ( ).

colors = ("red", "green", "blue")

print(type(colors))  # <class 'tuple'>

✔ Tuples are ordered but immutable.

🔹 4. range

Represents a sequence of numbers.

r = range(1, 5)

print(list(r))  # [1, 2, 3, 4]

3️⃣ Set Data Types

Used to store unique values.

🔹 1. set

Unordered collection of unique elements.

s = {1, 2, 3, 3}

print(s)  # {1, 2, 3}

✔ No duplicates allowed
✔ Unordered
✔ Mutable

🔹 2. frozenset

Immutable version of set.

fs = frozenset([1, 2, 3])

4️⃣ Mapping Data Type

🔹 dict (Dictionary)

Stores data in key-value pairs.

```
student = {
    "name": "Akhil",
    "age": 20
}
print(type(student))   # <class 'dict'>
```

✔ Mutable
✔ Keys must be unique

5️⃣ Boolean Data Type

🔹 bool

Stores only two values:

- True

- False

x = True

y = False

print(type(x))  # <class 'bool'>

Used in conditions:

print(5 > 3)  # True

6️⃣ None Data Type

🔹 NoneType

Represents no value.

x = None

print(type(x))  # <class 'NoneType'>

---

## Q9) Explain about Operators in Python.

## Ans) 🐍 Operators in Python

📌 Definition

Operators are special symbols used to perform operations on variables and values.

Example:

a = 5

```
b = 3

print(a + b)
```

Here, + is an operator used to add two numbers.

## 📚 Types of Operators in Python

Python mainly has 7 types of operators:

1. Arithmetic Operators
2. Relational (Comparison) Operators
3. Assignment Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

## 1️⃣ Arithmetic Operators

Used to perform mathematical calculations.

| Operator | Meaning | Example |
|---|---|---|
| + | Addition | 5 + 3 = 8 |
| - | Subtraction | 5 - 3 = 2 |

| | | |
|---|---|---|
| * | Multiplication | 5 * 3 = 15 |
| / | Division | 6 / 2 = 3.0 |
| % | Modulus (remainder) | 5 % 2 = 1 |
| ** | Exponent | 2 ** 3 = 8 |
| // | Floor division | 7 // 2 = 3 |

Example:

a = 10

b = 3

print(a % b)

Output:

1

## 2 Relational (Comparison) Operators

Used to compare two values.
The result is True or False.

| Operator | Meaning |
|---|---|
| == | Equal to |
| != | Not equal to |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal |
| <= | Less than or equal |

Example:

print(5 > 3)

Output:

True

## 3 Assignment Operators

Used to assign values to variables.

| Operator | Example |
|---|---|
| = | x = 5 |
| += | x+= 2 |
| -= | x-= 2 |
| *= | x*=2 |
| /= | x /= 2 |
| %= | x%= 2 |

Example:

x = 5

x += 3

print(x)

Output:

8

## 4 Logical Operators

Used to combine conditional statements.

| Operator | Meaning |
| --- | --- |
| and | True if both conditions are true |
| or | True if one condition is true |
| not | Reverses the condition |

Example:

print(5 > 3 and 10 > 8)

Output:

True

## 5 Bitwise Operators

Used to perform operations on binary numbers.

| Operator | Meaning |
|---|---|
| & | AND |
| ` | ` |
| ^ | XOR |
| ~ | NOT |
| << | Left shift |
| >> | Right shift |

Example:

```
print(5 & 3)
```

Output:

```
1
```

## 6 Membership Operators

Used to check whether a value exists in a sequence.

| Operator | Meaning |
|---|---|
| in | Value exists |
| not in | Value does not exist |

Example:

```
print(3 in [1,2,3,4])
```

Output:

```
True
```

## 7 Identity Operators

Used to check whether two variables refer to the same object.

| Operator | Meaning |
|---|---|
| is | Both variables refer to same object |
| is not | Variables refer to different objects |

Example:

a = 5

b = 5

print(a is b)

Output:

True

## 📊 Summary Table

| Type | Operators |
| --- | --- |
| Arithmetic | + - * / % ** // |
| Comparison | == != > < >= <= |
| Assignment | = += -= *= /= %= |
| Logical | and or not |
| Bitwise | `& |
| Membership | in, not in |

| | |
|---|---|
| Identity | is, is not |

### 📝 One-Line Exam Definition

Operators are special symbols used to perform operations on variables and values in Python.

---

## Q 10) Explain about Operator Precedence in Python.

### Ans) 🐍 Operator Precedence in Python

📌 Definition

Operator precedence refers to the priority of operators in an expression.
It determines which operation is performed first when multiple operators appear in the same expression.

Operators with higher precedence are evaluated before operators with lower precedence.

Here is Operator Precedence in Python (Highest to Lowest) in bullet points:

- () Parentheses
- ** Exponentiation
- +x , -x , ~x Unary operators

- * , / , // , % Multiplication, Division, Floor Division, Modulus
- + , - Addition, Subtraction
- << , >> Bitwise Shift Operators
- & Bitwise AND
- ^ Bitwise XOR
- | Bitwise OR
- == , != , > , < , >= , <= Comparison Operators
- not Logical NOT
- and Logical AND
- or Logical OR

📌 Important Point

- Operators at the top have higher precedence and are evaluated first.

📝 One-line exam statement

Operator precedence defines the order in which operators are evaluated in a Python expression.

# 1️⃣ Remember in Groups (Easiest Method)

Instead of memorizing 13 items, remember **5 groups**:

1. **Brackets** → ()
2. **Power** → **
3. **Unary** → +x , -x , ~x
4. **Arithmetic** → * / // % then + -
5. **Bitwise** → << >> → & → ^ → |
6. **Comparison** → == != > < >= <=
7. **Logical** → not → and → or

So the **simple flow** is:

**Brackets → Power → Unary → Arithmetic → Bitwise → Comparison → Logical**

## 2️⃣ Small Memory Sentence

You can remember it like this:

**"Brave People Use Arithmetic Before Comparing Logic."**

Meaning:

- **B** → Brackets
- **P** → Power

- **U** → Unary
- **A** → Arithmetic
- **B** → Bitwise
- **C** → Comparison
- **L** → Logical

---

# Q 11) Explain Conditional statements.

## Ans)1️⃣ if Statement

Syntax

```
if test_expression:
    statement(s)
```

Explanation

- The program checks the test expression (condition).
- If the condition is True, the statements inside if will run.
- If the condition is False, the statements inside if will not run.
- In Python, indentation (spaces at the beginning of a line) shows which statements belong to the if block.
- The block starts with indentation and ends when indentation stops.
- Python treats non-zero values as True.
- Python treats 0 and None as False.

Example Program

```
a = int(input("enter a value"))

b = int(input("enter b value"))

if a == b:

    print("yes")
```

What this program does

- It takes two numbers from the user.
- It checks whether a is equal to b.
- If they are equal, it prints yes.

## 2️⃣ if...else Statement

Syntax

```
if test_expression:

    body of if

else:

    body of else
```

Explanation

- The program checks the condition.
- If the condition is True, the if block runs.
- If the condition is False, the else block runs.
- Indentation is used to separate these blocks.

Example Program

Program to find the larger number

```
a = int(input("enter the value of a: "))

b = int(input("enter the value of b: "))

if a > b:

    large = a

else:

    large = b

print(large)
```

What this program does

- It asks the user to enter two numbers.
- It compares them.
- It stores the bigger number in large.
- Finally, it prints the larger number.

## 3️⃣ if...elif...else Statement

Syntax

```
if test_expression:

    body of if

elif test_expression:

    body of elif

else:
```

body of else

Explanation

- elif means else if.
- It is used when we want to check multiple conditions.
- If the if condition is False, Python checks the next elif.
- This continues until a True condition is found.
- If all conditions are False, the else block runs.
- Only one block will run.
- There can be many elif blocks, but only one else block.

Example Program

Check whether a character is a vowel

```
ch = input("enter any character: ")

if ch == "A" or ch == "E" or ch == "I" or ch == "O" or ch == "U":

    print(ch, "is a vowel")

elif ch == "a" or ch == "e" or ch == "i" or ch == "o" or ch == "u":

    print(ch, "is a vowel")

else:

    print(ch, "is not a vowel")
```

What this program does

- It takes a character from the user.
- It checks if the character is A, E, I, O, U (uppercase vowels).
- Then it checks a, e, i, o, u (lowercase vowels).
- If it matches, it prints vowel.

- Otherwise, it prints not a vowel.

### 4️⃣ Nested if Statements

Explanation

- Sometimes an if statement can be placed inside another if statement.
- This is called a nested if statement.
- Any number of if, elif, or else statements can be placed inside another.
- Indentation shows the level of nesting.
- Too many nested statements can make the program hard to read, so they should be used carefully.

---

## Q 12) Explain Loops in Python.

**Ans)** Here is your content rewritten in very simple English for a beginner, with clear bullet points.

### 1️⃣ for Loop

What is a for loop?

- A for loop is used to repeat a block of code.
- It works on a sequence like:
  - list
  - tuple
  - string

- Going through each item in a sequence is called iteration or traversal.

Syntax

```
for variable in sequence:
    body of loop
```

Explanation

- variable stores the value of each item in the sequence.
- The loop runs once for each item in the sequence.
- It stops when the last item is reached.
- Python uses indentation (spaces) to show the loop body.

Example: Sum of Numbers in a List

```
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]
sum = 0
for val in numbers:
    sum = sum + val
print("The sum is", sum)
```

What this program does

- It takes numbers from the list one by one.
- It keeps adding them to sum.
- Finally, it prints the total sum.

Output:

The sum is 48

## 2️⃣ Program to Check Perfect Number

What is a Perfect Number?

- A perfect number is a number that is equal to the sum of its factors (except itself).

Example:
6 → factors are 1, 2, 3

$1 + 2 + 3 = 6$

So 6 is a perfect number.

Program

```
n = int(input("Enter any number: "))
sum1 = 0
for i in range(1, n):
    if n % i == 0:
        sum1 = sum1 + i
if sum1 == n:
    print("The number is a Perfect number!")
else:
    print("The number is not a Perfect number!")
```

What this program does

- It checks all numbers from 1 to n-1.
- If the number divides n, it adds it to the sum.
- Finally, it checks if the sum equals n.

## 3️⃣ range() Function

What is range()?

- range() is used to generate a sequence of numbers.

Example:

range(10)

This produces numbers:

0 1 2 3 4 5 6 7 8 9

Different forms of range()

1️⃣ Only stop value

range(10)

Output → 0 to 9

2️⃣ Start and stop

range(2,10)

Output → 2 to 9

3️⃣ Start, stop, step

range(2,10,2)

Output → 2 4 6 8

Important Point

- range() does not store all numbers in memory.
- It generates numbers only when needed, which saves memory.

Example: Using range() with Index

genre = ['pop', 'rock', 'jazz']

```
for i in range(len(genre)):
    print("I like", genre[i])
```

Output:

I like pop

I like rock

I like jazz

## 4 for Loop with else

- A for loop can also have an else block.
- The else block runs after the loop finishes normally.
- If the loop stops because of a break statement, the else block will not run.

Example

digits = [0, 1, 5]

```
for i in digits:
    print(i)
else:
    print("No items left.")
```

Output:

```
0
1
5
No items left.
```

## 5️⃣ Program to Reverse a String

```
string1 = "pace student"
string2 = ""
i = len(string1) - 1
while i >= 0:
    string2 = string2 + string1[i]
    i = i - 1
print("original =", string1)
print("reverse =", string2)
```

What this program does

- It starts from the last character of the string.
- It adds characters one by one in reverse order.
- Finally, it prints the reversed string.

Output:

original = pace student

reverse = tneduts ecap

## 6 while Loop

What is a while loop?

- A while loop repeats a block of code as long as the condition is True.
- It is used when we do not know how many times the loop will run.

Syntax

```
while condition:
    body of loop
```

How it works

- Python checks the condition first.
- If the condition is True, the loop runs.
- After running once, Python checks the condition again.
- The loop stops when the condition becomes False.

Example: Sum of Natural Numbers

```
n = int(input("Enter n: "))
```

```
sum = 0

i = 1

while i <= n:

    sum = sum + i

    i = i + 1

print("The sum is", sum)
```

What this program does

- It adds numbers from 1 to n.
- Finally, it prints the total sum.

Example output:

```
Enter n: 10

The sum is 55
```

## 7 while Loop with else

- A while loop can also have an else block.
- The else block runs when the loop condition becomes False.
- If the loop stops because of a break statement, the else block will not run.

Example

```
counter = 0

while counter < 3:
```

```
    print("Inside loop")
    counter = counter + 1
else:
    print("Inside else")
```

Output:

```
Inside loop
Inside loop
Inside loop
Inside else
```

---

## Q 13) Explain about Control Statements.

### Ans) 1 break and continue Statements

- In Python, break and continue change the normal flow of a loop.
- Normally, a loop runs until its condition becomes False.
- But sometimes we want to:
  - Stop the loop immediately, or
  - Skip one iteration and continue the loop.
- For this purpose, Python provides break and continue statements.

## 2 break Statement

What is break?

- The break statement stops the loop immediately.
- After break, the program continues with the next statement after the loop.
- It does not check the loop condition again.

Important Points

- break terminates the loop completely.
- If break is used inside a nested loop (a loop inside another loop):
  - Only the inner loop stops.
  - The outer loop continues.

Syntax

```
break
```

Example

```
for val in "string":
    if val == "i":
        break
    print(val)
```

print("The end")

What this program does

- It goes through each letter of "string".
- When it finds i, the loop stops immediately.

Output:

s

t

r

The end

## 3️⃣ continue Statement

What is continue?

- The continue statement skips the current iteration.
- The loop does not stop, it moves to the next iteration.

Important Points

- continue skips the remaining code in that iteration.
- The loop continues normally with the next item.

Syntax

continue

Example

```
for val in "string":
    if val == "i":
        continue
    print(val)
print("The end")
```

What this program does

- It goes through each letter in "string".
- When it finds i, it skips printing it.
- The loop continues with the next letter.

Output:

```
s
t
r
n
g
The end
```

## 4 pass Statement

What is pass?

- pass is a null statement.
- It means do nothing.

Important Points

- It is used as a placeholder when a statement is required but no code is needed yet.
- The program does nothing when pass runs.
- A comment is ignored by Python, but pass is still a valid statement.

Syntax

pass

Example

```
for i in range(5):
    pass
```

What this program does

- The loop runs 5 times.
- But nothing happens because of pass.

---

## Q 14) Define Functions. Explain types of Functions with examples.

## Ans) FUNCTIONS:

Function is a block of reusable code that performs a specific task.

Instead of writing the same code again & again, we create a function & call it whenever needed.

ex:

```
def function_name():
    # return of function (optional)
    statement 1
    statement 2
    return value
```

def : Keyword used to define a function.

function_name : Name given to a function.

parameters : Input to the function (optional).

All statements inside the function must be indented.

TYPES OF FUNCTIONS

Functions are of two types:

- Built in functions
- User defined functions

---

BUILT-IN FUNCTIONS

• These are predefined functions provided by python.

• You can use them directly without defining them.

• These are already available.

• It saves time.

• Improves readability of code.

• No need to create them using def.

---

1) print()

Used to display the output.

Syntax:

print(variable_name)

ex:

name = "Akhil"

print(name)

Output:

Akhil

---

2) input()

Used to take input.

Syntax:

input()

ex:

name = input("Enter name :")

Output:

Enter name : Akhil

---

3) map()

Used to apply function to all elements in an iterable.

Syntax:

map(function, iterable)

ex:

num = ['1','2','3']

result = list(map(int, num))

print(result)

Output:

[1,2,3]

---

4) split()

Used to split a string into a list.

Syntax:

string.split(separator)

ex:

text = "Akhil is a handsome boy"

list = text.split()

print(list)

Output:

['A','k','h','i','l',' ','i','s',' ','a',' ','h','a','n','d','s','o','m','e',' ','b','o','y']

---

5) join()

Used to join elements of an iterable into string.

Syntax:

string.join(iterable)

ex:

name = ["Akhil","is","a","good","boy"]

result = " ".join(name)

print(result)

Output:

Akhil is a good boy

---

6) strip()

Used to remove space from beginning & ending of a string.

ex:

text = "  Python  "

print(text.strip())

Output:

Python

(no spaces)

---

7) replace()

Used to replace one substring into another substring.

Syntax:

text.replace(old, new)

ex:

text = "Akhil is very sweet"

print(text.replace("is","was"))

Output:

Akhil was very sweet

---

8) upper()

Convert all characters into uppercase.

Syntax:

text.upper()

ex:

text = "Akhil"

print(text.upper())

Output:

AKHIL

---

9) lower()

Convert all characters into lower case.

Syntax:

text.lower()

ex:

text = "AKHIL"

print(text.lower())

Output:

akhil

---

10) filter()

Used to filter elements of an iterable based on condition.

Syntax:

filter(function, iterable)

ex:

num = [1,2,3,4,5]

result = filter(lambda x : x%2==0 , num)

print(result)

Output:

[2,4]

---

11) enumerate()

Used to add a counter to elements of an iterable.

Syntax:

enumerate(iterable)

ex:

```
names = ["Akhil","Aditi","Rajini"]
for i,name in enumerate(names):
    print(i,name)
```

Output:

0 Akhil

1 Aditi

2 Rajini

---

12) zip()

Used to combine multiple iterables into pairs.

Syntax:

zip(iterable1 , iterable2)

ex:

```
name = ["Akhil","Aditi","Rajini"]
marks = [80,90,100]
result = list(zip(name,marks))
print(result)
```

Output:

[("Akhil",80),("Aditi",90),("Rajini",100)]

---

13) any()

Returns True if atleast one element satisfies the given condition.

ex:

values = [0, False, 5]

print(any(values))

Output:

True

---

14) all()

Used to return True if all elements satisfies the given condition.

ex:

values = [1,2,3]

print(all(values))

Output:

True

---

15) sorted()

Used to sort elements in an iterable.

Syntax:

sorted(iterable)

ex:

values = [5,7,1]

print(sorted(values))

Output:

[1,5,7]

---

USER-DEFINED FUNCTIONS

User-defined function is a function created by the programmer to perform a specific task.

- It is defined by using def keyword.
- It improves readability of code.
- Easy to evaluate the error.
- Reduce program size.

Syntax:

def function_name(parameters):

statements

return value

(function definition / declaration)

ex:

def addition(a,b):

return a+b

(function definition)

result = addition(2,3)

print(result)

(function calling)

argument

Output:

5

---

## Q 15) Explain Recursive Functions.

**Ans)** Here is a simple, exam-ready explanation of Recursive Functions in Python in bullet points.

### Recursive Functions in Python

Definition

- A recursive function is a function that calls itself to solve a problem.
- It keeps calling itself until a stopping condition is reached.

## Important Parts of Recursion

- Recursive Case
  - The part where the function calls itself.
- Base Case
  - The stopping condition of the recursion.
  - Without a base case, the function will run forever (infinite recursion).

## Syntax

```
def function_name(parameters):
    if base_condition:
        return value
    else:
        return function_name(smaller_problem)
```

## Example: Factorial Using Recursion

Factorial of a number:

[
n! = n \times (n-1) \times (n-2) \times ... \times 1
]

Example:
$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Program

```
def factorial(n):
    if n == 1:          # Base case
        return 1
    else:
        return n * factorial(n-1)   # Recursive call
print(factorial(5))
```

Output

120

## How the Program Works

factorial(5)

$= 5 \times$ factorial(4)

$= 5 \times 4 \times$ factorial(3)

$= 5 \times 4 \times 3 \times$ factorial(2)

$= 5 \times 4 \times 3 \times 2 \times \text{factorial}(1)$

$= 5 \times 4 \times 3 \times 2 \times 1$

$= 120$

### Advantages of Recursion

- Makes some problems easier to solve.
- Code becomes shorter and cleaner.
- Useful in tree structures, searching, and mathematics.

### Disadvantages of Recursion

- Uses more memory because each function call is stored.
- Can become slow for large problems.
- If the base case is missing, it causes infinite recursion.

### One-line Exam Definition

A recursive function is a function that calls itself repeatedly until a base condition is reached.

---

## Q 16) Explain Lambda Functions.

**Ans)** Lambda Functions in Python

1️⃣ Definition

- A lambda function is a small anonymous function in Python.

- It is created using the lambda keyword.
- It can take any number of arguments but contains only one expression.
- The result of the expression is automatically returned.

2️⃣ Syntax

lambda arguments : expression

Explanation

- lambda → keyword used to create a lambda function
- arguments → input values for the function
- expression → operation performed and returned automatically

## 3️⃣ Example

square = lambda x: x * x

print(square(5))

Output

25

Explanation

- x is the argument.
- x * x is the expression.
- The function returns the square of the number.

## 4️⃣ Lambda Function with One Argument

Example

square = lambda x: x * x

print(square(4))

Output

16

Explanation

- The lambda function takes one argument (x).
- It returns x multiplied by itself.

## 5️⃣ Lambda Function with Two Arguments

Example

add = lambda a, b: a + b

print(add(3, 5))

Output

8

Explanation

- The lambda function takes two arguments (a and b).
- It returns the sum of the two numbers.

## 6 Lambda Function with Three Arguments

Example

multiply = lambda a, b, c: a * b * c

print(multiply(2, 3, 4))

Output

24

Explanation

- The lambda function takes three arguments (a, b, c).
- It returns the product of the three numbers.

## Important Points

- Lambda functions are anonymous (no name).
- They are used for small and simple operations.
- They contain only one expression.
- They do not use the return keyword.

## One-Line Exam Answer

A lambda function is a small anonymous function defined using the lambda keyword that can take any number of arguments but contains only one expression.

**lambda useful in specialized roles:**

**1.** lambda is an expression, not a statement. Because of this, a lambda can appear in places a def is not allowed. For example, places like inside a list literal, or a function call's arguments. As an expression, lambda returns a value that can optionally be assigned a name. In contrast, the def statement always assigns the new function to the name in the header, instead of returning is as a result.

**2**. lambda's body is a single expression, not a block of statements. The lambda's body is similar to what we'd put in a def body's returnstatement. We simply type the result as an expression instead of explicitly returning it. Because it is limited to an expression, a lambda is less general that a def. We can only squeeze design, to limit program nesting. lambda is designed for coding simple functions, and def handles larger tasks.

---

## Q 17) Explain Pass by value and Pass by Reference.

**Ans)** In Python, the idea of reference vs value comes from how variables store data. Variables usually store a reference (address) to an object in memory, not the actual value itself.

---

### 1. Pass by Value

In pass by value, a copy of the value is sent to the function.

Any changes inside the function do not affect the original variable.

Example

```
def change(x):
    x = x + 5
    print("Inside function:", x)
num = 10
change(num)
print("Outside function:", num)
```

Output

```
Inside function: 15
Outside function: 10
```

Explanation

num = 10

Function receives a copy of 10

Inside function it becomes 15

Original variable num remains 10

So the original value is not changed.

---

## 2. Pass by Reference

In pass by reference, the address (reference) of the variable is passed to the function.

So the function can modify the original data.

Example

```
def change(lst):
    lst.append(4)

numbers = [1,2,3]

change(numbers)

print(numbers)
```

Output

```
[1, 2, 3, 4]
```

Explanation

numbers list is passed by reference

Function modifies the same list in memory

So the original list changes

---

Important Note about Python

Python actually uses something called Pass by Object Reference (Call by Sharing).

This means:

Immutable objects behave like pass by value

int

float

string

tuple

Mutable objects behave like pass by reference

list

dictionary

set

---

Simple Table

TypeBehavior Example

Immutable Pass by value behavior int, float, string

Mutable Pass by reference behavior list, dict, set

---

✅ In one line:

Python passes references to objects, but immutable objects cannot be modified, so they behave like pass by value, while mutable objects behave like pass by reference.

---

## Q 18) Explain Function Arguments in Python.

**Ans)** Function Arguments in Python

Function arguments are the values passed to a function when it is called.

They allow the function to receive data and perform operations on it.

A function may have zero, one, or multiple arguments.

---

Syntax

def function_name(parameters):

    statements

When calling the function:

function_name(arguments)

Parameters → Variables in the function definition

Arguments → Values passed when calling the function

---

Example

```
def addition(a, b):
    return a + b
result = addition(5, 3)
print(result)
```

Output

8

Explanation

a and b → parameters

5 and 3 → arguments

---

Types of Function Arguments in Python

There are four main types of arguments.

1. Positional Arguments

2. Keyword Arguments

3. Default Arguments

4. Variable-Length Arguments

---

1. Positional Arguments

In positional arguments, the order of arguments matters.

Example

```
def student(name, age):
    print(name, age)
student("Akhil", 20)
```

Output

Akhil 20

Explanation

"Akhil" is assigned to name

20 is assigned to age

If the order changes, the output changes.

```
student(20, "Akhil")
```

Output

20 Akhil

---

2. Keyword Arguments

In keyword arguments, values are passed using parameter names.

Order does not matter.

Example

```
def student(name, age):
```

```
    print(name, age)
student(age=20, name="Akhil")
```

Output

Akhil 20

Explanation

Python matches values using parameter names.

---

3. Default Arguments

Default arguments have a predefined value.

If the user does not provide a value, the default value is used.

Example

```
def greet(name="Guest"):
    print("Hello", name)
greet()
greet("Akhil")
```

Output

Hello Guest

Hello Akhil

Explanation

If no argument is given → default value is used.

---

4. Variable-Length Arguments

Used when we do not know how many arguments will be passed.

Two types exist.

(a) *args

Allows multiple positional arguments.

```
def add(*numbers):
    print(sum(numbers))
add(1,2,3)
add(4,5,6,7)
```

Output

6

22

---

(b) **kwargs

Allows multiple keyword arguments.

```
def student(**details):
    print(details)
```

student(name="Akhil", age=20, city="Hyderabad")

Output

{'name': 'Akhil', 'age': 20, 'city': 'Hyderabad'}

---

Summary Table

Argument Type Description

Positional Order of arguments matters

Keyword Arguments passed using parameter names

Default Parameters have default values

Variable Length Accepts unlimited arguments (*args, **kwargs)

---

✅ In simple terms:

Function arguments allow us to pass data into functions so they can perform tasks using that data.

---

## Q 19) Explain NameSpacing and Scoping.

**Ans)**Namespaces and Scope using LEGB Rule

1️⃣ Namespace

- A namespace is a place where variable names and their values are stored.
- It works like a dictionary where:
  - Variable name → Key
  - Value/Object → Value

Example idea:

x → 10

name → "Akhil"

Python uses namespaces to avoid conflicts between variable names.

## 2️⃣ Scope

- Scope means the area of the program where a variable can be used.
- Python decides which variable to use based on the LEGB rule.

## 3️⃣ LEGB Rule

Python searches for variables in the following order:

- L → Local
- E → Enclosed
- G → Global
- B → Built-in

Python checks each scope one by one until it finds the variable.

## 4 Local Scope (L)

- Variables defined inside a function.
- These variables can be used only inside that function.
- Python checks local scope first.

Example:

```
def test():
    x = 5
    print(x)
test()
```

Here x is a local variable.

## 5 Enclosed Scope (E)

- Variables defined in the outer function of a nested function.
- The inner function can access variables from the outer function.

Example:

```
def outer():
    x = 10
    def inner():
        print(x)
```

```
    inner()

outer()
```

Here x belongs to the enclosing function.

## 6 Global Scope (G)

- Variables defined outside all functions.
- They can be accessed anywhere in the program.

Example:

```
x = 20

def test():

    print(x)

test()
```

Here x is a global variable.

## 7 Built-in Scope (B)

- These are names already defined by Python.
- Examples include functions like:

```
print()

len()

type()
```

sum()

Python checks the built-in scope last.

## 8️⃣ Summary of Variable Search

When Python sees a variable, it searches in this order:

1. Local scope
2. Enclosed scope
3. Global scope
4. Built-in scope

If the variable is not found in any scope, Python gives a NameError.

## Easy Way to Remember

LEGB

- L → Local
- E → Enclosed
- G → Global
- B → Built-in

## One-line Exam Definition

Python follows the LEGB rule (Local, Enclosed, Global, Built-in) to determine the scope and namespace where a variable is searched.

---

# CODING ON PATTERN PROBLEMS:

```
#Pattern problems

#1. Right-Angled Triangle Pattern

'''Expected Output:

*

**

***

****

*****'''

'''for i in range(1,6):

    for j in range(1,i+1):

        print("*",end="")

    print()'''



#Q2) 2. Inverted Right-Angled Triangle

'''Expected Output:

*****

****

***

**

*'''
```

```
'''n=6
for i in range(1,6):
    for j in range(1,n):
        print("*",end="")
    n-=1
    print()'''


#Q3) 3. Pyramid Pattern
'''Expected Output:
    *
   ***
  *****
 *******
*********'''
'''n=6
for i in range(1,6):
    for j in range(1,n):
        print(" ",end="")
    n-=1
    for k in range(1,2*(i)):
        print("*",end="")
    print()'''
```

```
#Q4) 4. Inverted Pyramid Pattern
'''Expected Output:
*********
 *******
  *****
   ***
    *'''
'''n=0
m=9
for i in range(1,6):
    for j in range(1,n+1):
        print(" ",end="")
    n+=1
    for k in range(1,m+1):
        print("*",end="")
    m-=2
    print()'''

#5. Diamond Pattern
'''Expected Output:
    *
```

```
  ***
 *****
*******
*********
*******
 *****
  ***
   *'''
'''n=5
for i in range(1,5):
    for j in range(1,n):
        print(" ",end="")
    n-=1
    for k in range(1,2*i):
        print("*",end="")
    print()
#reverse
p=0
q=10
for i in range(1,6):
    for j in range(1,p+1):
        print(" ",end="")
```

```
    p+=1
    for k in range(1,q):
        print("*",end="")
    q-=2
    print()'''


#6. Hollow Square Pattern
'''Expected Output:
*****
*   *
*   *
*   *
*****'''
'''for i in range(1,6):
    if i==1 or i==5:
        for j in range(1,6):
            print("*",end="")
        print()
    else:
        for k in range(1,6):
            if k==1 or k==5:
                print("*",end="")
```

```
        else:
            print(" ",end="")
    print()'''

#7. Hollow Triangle Pattern
'''Expected Output:
*
**
* *
*  *
*****'''
'''for i in range(1,6):
  if i==3 or i==4:
    for j in range(1,i+1):
      if j==1 or j==i:
        print("*",end="")
      else:
        print(" ",end="")
    print()
  else:
    for j in range(1,i+1):
      print("*",end="")
```

```
    print()'''


#8. Mirrored Right Triangle
'''Expected Output:
   *
  **
 ***
****
*****'''
'''n=4
for i in range(1,6):
   for j in range(1,n+1):
      print(" ",end="")
   n-=1
   for k in range(1,i+1):
      print("*",end="")
   print()'''


#9. Downward Triangle
'''Expected Output:
*****
 ****
```

```
 ***
  **
   *'''
'''n=5
for i in range(1,6):
    for k in range(1,i):
        print(" ",end="")
    for j in range(1,n+1):
        print("*",end="")
    print()
    n-=1'''


#10. X Shape Pattern
'''Expected Output:
*   *
 * *
  *
 * *
*   *'''
'''for i in range(1,6):
    for j in range(1,6):
        if i==j or i+j==6:
```

```
            print("*",end=" ")
        else:
            print(" ",end="")
    print()'''


#11. Plus (+) Pattern
'''Expected Output:
  *
  *
*****
  *
  *'''
'''for i in range(1,6):
    if i==3:
        for j in range(1,6):
            print("*",end="")
        print()
    else:
        for j in range(1,3):
            print(" ",end="")
        print("*")'''
```

```
#12. Hollow Rectangle

'''Expected Output:

******

*    *

*    *

*    *

******'''

'''for i in range(1,6):

   if i==1 or i==5:

      for j in range(1,6):

         print("*",end="")

      print()

   else:

      for j in range(1,6):

         if j==1 or j==5:

            print("*",end="")

         else:

            print(" ",end="")

      print()'''



#13. Increasing Stars by Two

'''Expected Output:
```

```
*

***

*****

*******

*********'''

'''n=1

for i in range(1,6):

    for j in range(1,n+1):

        print("*",end="")

    n+=2

    print()'''



#14. Cross Pattern

'''Expected Output:

*   *

 * *

  *

 * *

*   *'''

'''for i in range(1,6):

    for j in range(1,6):

        if j==i or i+j==6:
```

```
            print("*",end="")
        else:
            print(" ",end="")
    print()'''


#15. Letter H Pattern
'''Expected Output:
*   *
*   *
*****
*   *
*   *'''
'''for i in range(1,6):
    if i==3:
        for j in range(1,6):
            print("*",end="")
        print()
    else:
        for j in range(1,6):
            if j==1 or j==5:
                print("*",end="")
            else:
```

```
            print(" ",end="")

        print()'''
```

---

# CODING PROBLEMS ON RECURSIONS:

```
#Recursions Problems



#Q1) Write a recursive function to find the factorial of a number.

'''def fact(n):

    if n==0 or n==1:

        return 1

    else:

        return n*fact(n-1)

print(fact(5))'''

'''

Output:

120

'''



#Q2) Write a recursive program to compute the nth Fibonacci number.

'''def fib(n):

    if n==0:
```

```
        return 0
    elif n==1:
        return 1
    else:
        return fib(n-1)+fib(n-2)
n=int(input("Enter n:"))
print(fib(n))'''
'''
Output:
Enter n:6
8
'''


#Q3) Write a recursive function to find the sum of first n natural numbers.
'''def add(n):
    if n==0:
        return 0
    else:
        return n+add(n-1)
print(add(5))'''
'''
Output:
```

```
15
'''


#Q4) Write a recursive function to calculate the power of a number (x^n).
'''def power(a,n):
    if n==1:
        return a
    if n==0:
        return 1
    else:
        return a*power(a,n-1)
print(power(2,5))'''
'''
Output:
32
'''


#Q5) Write a recursive program to reverse a string.
'''def rev(text):
    if len(text)==0:
        return text
    else:
```

```
        return rev(text[1:]) + text[0]

text="Akhil"

print(rev(text))'''

'''

Output:

lihkA

'''



#Q6) Write a recursive function to print numbers from 1 to n.

'''def nat(n):

    if n==1:

        print(1)

        return

    else:

        nat(n-1)

        print(n)

print(nat(5))'''

'''

Output:

1

2

3
```

```
4

5

'''



#Q7) Write a recursive function to find the sum of digits of a number.

'''def add(n):

    if n==0:

        return n

    else:

        return n%10 + add(n//10)

print(add(256))'''

'''

Output:

13

'''



#Q8) Write a recursive function to count the number of digits in a number.

'''def dis(n,count):

    if n==0:

        return count

    else:

        return dis(n//10,count+1)
```

```
print(dis(256,0))'''
'''
Output:
3
'''


#Q9) Write a recursive function to find the product of digits of a number.
'''def pod(n):
    if n==0:
        return 1
    else:
        return n%10 * pod(n//10)
print(pod(123))'''
'''
Output:
6
'''


#Q10) Write a recursive function to print numbers from n to 1.
'''def dis(n):
    if n==0:
        return
```

```
    else:
        print(n)
        return dis(n-1)
print(dis(5))'''
'''
Output:
5
4
3
2
1
'''

#Q11) Write a recursive function to find the maximum element in a list.
'''def maxi(Ls):
    if len(Ls)==1:
        return Ls[0]
    max_rest=maxi(Ls[1:])
    if Ls[0]>max_rest:
        return Ls[0]
    else:
        return max_rest
```

```
print(maxi([1,3,2,5,7]))'''
'''
Output:
7
'''


#Q12) Write a recursive function to find the minimum element in a list.
'''def mini(Ls):
    if len(Ls)==1:
        return Ls[0]
    min_rest=mini(Ls[1:])
    if Ls[0]<min_rest:
        return Ls[0]
    else:
        return min_rest
print(mini([3,4,1,9]))'''
'''
Output:
1
'''


#Q13) Write a recursive program to check if a string is a palindrome.
```

```
'''def pal(text):
    if len(text)==0:
        return text
    else:
        return text[0] + pal(text[1:])
name=input("Enter name: ")
if name==pal(name):
    print("Palindrome")
else:
    print("Not a palindrome")'''
'''
output:
Enter name: madam
Palindrome
'''


#Q14) Write a recursive function to count vowels in a string.
'''def vow(text,count):
    if len(text)==1:
        if text[0]=='a' or text[0]=='e' or text[0]=='i' or text[0]=='o' or text[0]=='u':
            count+=1
        return count
```

```
    else:
        if text[0]=='a' or text[0]=='e' or text[0]=='i' or text[0]=='o' or text[0]=='u':
            count+=1
            #return count
        return vow(text[1:],count)
text="akhil"
count=0
print(vow(text,count))'''
'''
Output:
2
'''


#Q15) Write a recursive function to count occurrences of an element in a list.
'''def occ(Ls,x,count):
    if len(Ls)==1:
        if Ls[0]==x:
            count+=1
        return count
    else:
        if Ls[0]==x:
            count+=1
```

```
        return occ(Ls[1:],x,count)

print(occ([1,2,3,3],3,0))'''

'''

Output:

2

'''



#Q16) Write a recursive function to sum all elements in a list

'''def add(Ls):

    if len(Ls)==1:

        return Ls[0]

    else:

        return Ls[0] + add(Ls[1:])

print(add([1,2,3,4]))'''

'''

Output:

10

'''



#Q17) Write a recursive function to multiply two numbers without using '*'.

'''def mult(a,b,count):

    if count==b-1:
```

```
        return a
    else:
        return a + mult(a,b,count+1)
print(mult(3,7,0))'''
'''
Output:
21
'''


#Q18) Write a recursive function to find GCD of two numbers.
'''def gcd(a,b):
    if b==0:
        return a
    else:
        return gcd(b,a%b)
print(gcd(12,8))'''
'''
Output:
4
'''


#Q19) Write a recursive function to calculate the length of a string.
```

```
'''def length(text,count):
    if len(text)==0:
        return count
    else:
        return length(text[1:],count+1)
print(length("Akhil",0))'''
'''
Output:
5
'''


#Q20) Write a recursive function to generate Fibonacci series up to n terms.
'''def fib(n):
    if n==0:
        return 0
    if n==1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
n=8
for i in range(n):
    print(fib(i))'''
```

```
'''
Output:
0
1
1
2
3
5
8
13
'''


#Q21) Write a recursive function to find LCM of two numbers using GCD.
'''def gcd(a,b):
    if b==0:
        return a
    else:
        return gcd(b,a%b)
print(2*3/gcd(2,3))'''
'''
Output:
6.0
```

```
'''



#Q22) Write a recursive program to check if an array is sorted.

'''def sort(arr):
    if len(arr)==1:
        return True
    if arr[0]>arr[1]:
        return False
    return sort(arr[1:])


print(sort([1,2,3,4]))'''

'''

Output:

True

'''



#Q23) “Write a recursive function to find the first occurrence of an element in a list.”

'''def occ(Ls,a,count):
    if len(Ls)==1:
        if Ls[0]==a:
            return count
    if Ls[0]==a:
```

```
        return count
    return occ(Ls[1:],a,count+1)
print(occ([1,3,3,3],3,0))'''
'''
Output:
1
'''


#Q24) Write a recursive function to find the last occurrence of an element in a list.
'''def loc(Ls,a,occ,count,ind):
    if len(Ls)==1:
        if Ls[0]==a:
            occ.append(ind)
            count+=1
            return occ[count-1]
        else:
            return occ[count-1]
    if Ls[0]==a:
        count+=1
        occ.append(ind)
    return loc(Ls[1:],a,occ,count,ind+1)
print(loc([1,2,3,3,3,3],3,[],0,0))'''
```

```
'''

Output:

5

'''



#Q25) Write a recursive function to count the number of zeros in an integer

'''def zeros(Ls,count):

    if len(Ls)==1:

        if Ls[0]==0:

            count+=1

            return count

        else:

            return count

    if Ls[0]==0:

        count+=1

    return zeros(Ls[1:],count)

print(zeros([1,2,0,0,0],0))'''

'''

Output:

3

'''
```

```
#Q26) Write a recursive function to remove all occurrences of a character from a string.

'''def rem(text,a):
    if a not in text:
        return text
    else:
        text=text.replace(a,'',1)
    return rem(text,a)
print(rem("Akkil","k"))'''
'''
Output:
Ail
'''


#Q27) Write a recursive function to replace all spaces in a string with '-'.

'''def space(text):
    if ' ' not in text:
        return text
    else:
        text=text.replace(' ','-',1)
    return space(text)
print(space('Akhil is a good boy'))'''
'''
```

```
Output:

Akhil-is-a-good-boy

'''



#Q28) Write a recursive function to print all elements of a list in reverse order.

'''def rev(Ls):

    if len(Ls)==1:

        print(Ls[0])

        return

    print(Ls[len(Ls)-1])

    Ls.pop()

    return rev(Ls)

print(rev([1,2,3,4]))'''

'''

Output:

4

3

2

1

'''
```

```
#Q29) Write a recursive function to calculate the sum of squares of first n numbers.

'''def squ(n):

   if n==1:

      return 1

   else:

      return n**2 + squ(n-1)

print(squ(5))'''

'''

Output:

55

'''
```

---

# CODING PROBLEMS ON MATCH CASES:

```
#1)

'''mon=int(input("Enter mon num:"))

match mon:

   case 1:

      print("Jan")

   case 2:

      print("Feb")
```

```
case 3:
    print("Mar")
case 4:
    print("Apr")
case 5:
    print("May")
case 6:
    print("Jun")
case 7:
    print("July")
case 8:
    print("Aug")
case 9:
    print("Sep")
case 10:
    print("Oct")
case 11:
    print("Nov")
case 12:
    print("Dec")
case _:
    print("Invalid")'''
```

```
#2)
'''day=int(input("Enter day:"))
match day:
   case 1:
      print("Sunday")
   case 2:
      print("Monday")
   case 3:
      print("Tuesday")
   case 4:
      print("Wed")
   case 5:
      print("Thu")
   case 6:
      print("Fri")
   case 7:
      print("sat")
   case _:
      print("Invalid")'''

#3)
```

```
'''char=input("eNTER alphabet:")
match char:
    case 'a' | 'e' | 'i' | 'o' | 'u':
        print("Vowel")
    case _:
        print("Consonent")'''


#4)
'''grade=input("Enter grade:")
match grade:
    case 'A':
        print("Grade A")
    case 'B':
        print("Grade B")
    case 'C':
        print("Grade C")
    case 'D':
        print("Grade D")
    case _:
        print("Invalid")'''


#5)
```

```
'''num=int(input("Enter num:"))
match num:
  case 1:
    print("One")
  case 2:
    print("Two")
  case 3:
    print("Three")
  case 4:
    print("four")
  case 4:
    print("five")
  case _:
    print("Invalid")'''


#6)
'''num=int(input("Enter num:"))
match num:
  case 1:
    print("Apple")
  case 2:
    print("Mango")
```

```
    case 2:
        print("Banana")
    case _:
        print("Invalid")'''

#7)'
'''num=int(input("Enter num:"))
match num:
    case 1:
        print("Tea")
    case 2:
        print("Coffee")
    case 2:
        print("Milk")
    case _:
        print("Invalid")'''

#8)
'''num=int(input("Enter num:"))
count=0
while num>0:
    d=num%10
```

```
    count+=1
    num=num//10
match count:
    case 1:
        print("1 digit")
    case 2:
        print("2 digit")
    case 3:
        print("3 digit")'''


#9)
'''color=input("Enter color:")
ch=color.lower()
match ch:
    case "red":
        print("Stop")
    case "green":
        print("Move")
    case "yellow":
        print("Redy to move or stop")
    case _:
        print("iNvalid")'''
```

```
#10)

'''num=int(input("Enter num:"))

match num:

   case 1:

      print("Summer")

   case 2:

      print("Rainy")

   case 3:

      print("Winter")

   case _:

      print("Invalid")'''
```

---